

Photovoltaic-Panel Assembly Cell

Rapid Project

Done by:

Alejandro Ruiz, Marcos Granados, Hugo Cárdenas

Index

Project Definition.....	4
Implementation Approach.....	4
Results.....	4
Scalability and Future Developments.....	5
Project Specifications - A.....	5
A.1 - Choose and describe the manufacturing process.....	5
A.2 - Justify each selected robot based on values in its datasheet.....	6
A.3 - Use at least two TCPs other than "tool0" for each robot used.....	7
A.4 - Define at least two work objects other than "wobj0".....	8
A.5 - Capture by guidance of only one point for each work object defined.....	8
A.6 - Use approaching points to bring the tool closer to objects.....	8
A.7 - Use the appropriate movement instructions.....	9
A.9 - Use at least one parameter-passing routine.....	9
A.10 - Justify whether it is reasonable for the process to be carried out robotically instead of manually.....	9
Project Specifications - B.....	10
B - Functional summary.....	10
Robots:.....	10
Regarding tools:.....	11
Optimization of procedures:.....	11
Digital Signals.....	11

Project Definition

This project was developed to solve a specific industrial requirement: The automation of the assembly of photovoltaic panels onto structural metal frames through precise, robotic processes.

Manual handling of long metal profiles and fragile photovoltaic panels introduces unacceptable levels of error and inefficiency. Our goal was to design a process where industrial robots perform every critical operation, ensuring both precision and consistency.

We've worked in this process in a way that it can be repeated in several lines which can lead to the build of wider photovoltaic panels, following the clients needs.

Implementation Approach

The assembly cell was configured using two ABB IRB 2600 robots, these specific models were chosen due to its reach and payload capabilities, being 12kg and 20kg to ensure the stability and need throughout the robotic process.

Robot 1 collects metal profiles from a conveyor and assembles them into a rectangular frame using a dedicated fixture. Robot 1 use precision alignment is achieved entirely through robotics manipulation, Robot 2 completes then the assembly by soldering.

All operations were modeled and simulated using ABB RobotStudio, and control logic was exclusively written in RAPID. Every SmartComponent was designed to reflect the exact dimensions and tolerances of the materials used, avoiding reliance on abstract patterns or assumptions.

Results

The project delivered a fully functional simulation of the photovoltaic assembly line, capable of immediate deployment and scalable to meet higher production volumes.

The solution is stable and supports direct replication for creating additional production lines. Under current simulation parameters.

Scalability and Future Developments

This system was intentionally designed for rapid scalability. Using the same RAPID codebase and simulation configurations, additional production lines can be brought online to multiply output without any structural changes to the system.

This project could be improved following clients needs for wider or larger photovoltaic panels, simulating and implementing downstream integration for the assembly of larger photovoltaic arrays using the produced modules.

This solution is a concrete, immediately applicable system designed for industrial deployment. It establishes a scalable foundation for efficient, high-precision photovoltaic panel manufacturing and is fully prepared for integration into broader renewable energy production strategies.

Project Specifications - A

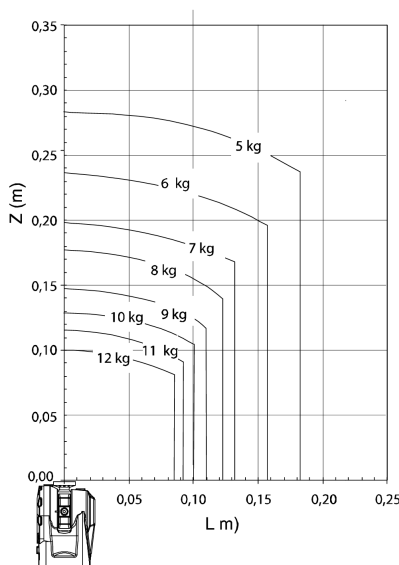
A.1 - Choose and describe the manufacturing process

We have chosen the assembly of a solar panel as our project, using two robots. Both robots have two tools: one for gripping and the other for welding.

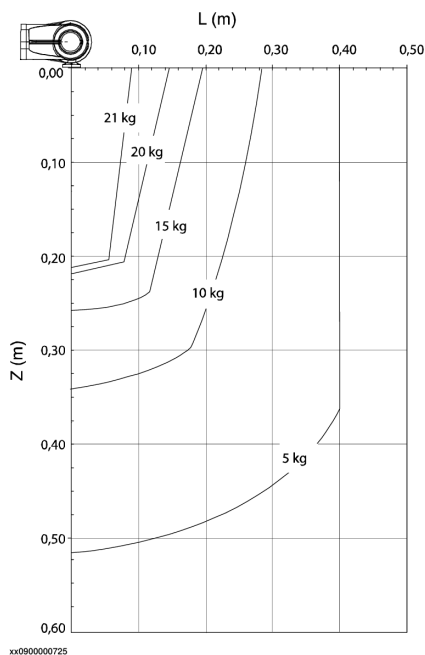
Our project consists of three elements: two metal bars and a solar panel. These bars will exit on the conveyor belt, and we will proceed to pick them up and place them in their positions. The solar panel will then exit on the other conveyor belt. We will pick up this plate and place it on top of these two bars, then continue with the welding process. The first robot will weld a portion of each bar to join it with the plate, and the second robot will finish welding the pieces. After finishing welding, the second robot will pick up the plate and place it on the last conveyor belt.

A.2 - Justify each selected robot based on values in its datasheet

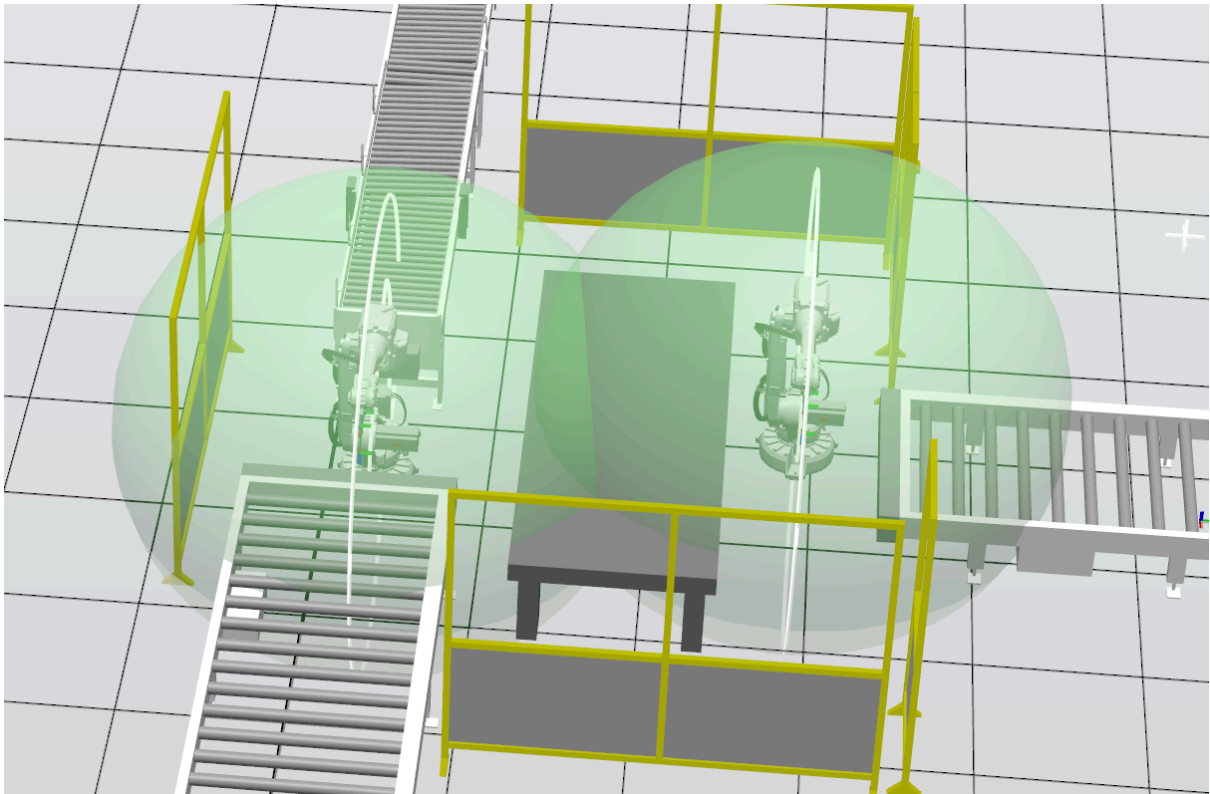
We chose the ABB IRB 2600-20/1.65 and ABB IRB 2600-12/1.65. The robots have six degrees of freedom, which allow them to perform precise movements for both positioning and soldering components, with all axes required to correctly orient the soldering tool. It is redundant, meaning it can reach the same position through different configurations.



The first one only requires you to manipulate objects that weigh less than 12 kg in a short range, which is possible according to the load diagram provided in the specifications. When the robot reaches a point further away, it does not carry objects, it does it for welding.



Similarly, the second robot reaches for far points only to weld, not when carrying an object. And when it does, it only moves it within a range in which it does not lose load capability. The selected robot has a payload of 20, because it should be able to carry all the components together, once the set has been welded.



The two robots share a section of work space, therefore it is mandatory to implement synchronization mechanisms to avoid collision.

A.3 - Use at least two TCPs other than "tool0" for each robot used

In our project, we used two different Tool Center Points (TCPs) for each robot, allowing us to perform specific tasks during the assembly process.

TCP 1 – Grip: We called this tool grip and it is designed to perform gripping operations. It allows the robots to hold both the metal bars and the solar panel, allowing for their transport and precise positioning on the structure. Later, it is also used to grab the whole set and reposition it.

TCP 2 – Pen: The second tool, called pen, is used to carry out the welding process. This tool simulates the welding process. **IMPORTANT:** Note that the program glitches sometimes and loses the correct position of the TCP 2 in the first robot controller. If this happened, modify the definition of the tool to: TASK PERS tooldata

```
Pen:=[TRUE,[[170.754,0.18,190.741],[0.92388,0,0.382683,0]],1,[0,0,1],[1,0,0,0],0,0,0];
```

A.4 - Define at least two work objects other than "wobj0"

In our robot programming, we defined two Work Objects (Wobj), called Table, one for each robot. These work objects represent reference points in physical space that facilitate precise operations, especially during the welding phase.

Each Wobj Table is located at a point near the edge of the table. This configuration allows both robots to have their own frame of reference, adapted to their work area, ensuring better coordination and precision during assembly.

Thanks to the use of these Wobjs, robots can perform more precise movements when welding, avoiding alignment errors and ensuring a firm connection between the parts.

A.5 - Capture by guidance of only one point for each work object defined

In our program, we only capture one or two base points for each work object using guidance, such as the point where we pick up the metal bar. From that point, we use the Offs() function in RAPID to move to relative positions, such as the placement or welding point. This way, we don't need to manually capture all the points, and we can build the entire path using relative transformations, which facilitates programming and makes the system more flexible to physical changes.

A.6 - Use approaching points to bring the tool closer to objects

In our program, we use approach points to ensure the tool safely approaches objects before interacting with them. For example, before grabbing a metal bar, we define a point for the robot to first position above and then lower in a straight line. This prevents collisions and allows for a controlled entry. We apply the same approach before placing the solar panel or performing the weld, using positive vertical (Z) offsets to approach and negative offsets to retreat. This ensures smooth and safe movements throughout the process.

A.7 - Use the appropriate movement instructions

In our program, we use different types of movement instructions depending on the needs of each operation. For movements between targets, where trajectory linearity is not required, we use joint movements (MoveJ), which are faster and more efficient. For operations requiring precision, such as gripping parts, placing solar panels, or welding, we use linear movements (MoveL), which guarantee straight and safe trajectories. Velocity is also modified depending on the instruction required, as it is the precision parameter that can go down to z0 or fine for the most precise movements.

A.8 Flow of the program

In our case, the program flow is linear because we designed the system with a repetitive industrial process in mind, where all parts (metal bars and solar panels) have fixed, standardized dimensions. This decision is based on the logic of optimizing the mass production of solar panels, where repeatability improves efficiency, reduces cycle times, and minimizes errors. Therefore, we don't consider it necessary to condition the flow based on internal variables such as part size, as the process is designed to operate continuously and stably under constant conditions.

A.9 - Use at least one parameter-passing routine

In our program, we use PROC-type routines to structure repetitive actions such as component gripping. We define two functions: grab_bar_func and grab_plate_func, both designed to execute the gripping sequence in a modular manner.

A.10 - Justify whether it is reasonable for the process to be carried out robotically instead of manually

Based on the "Implementation Criteria" theory, it is not reasonable for the process to be carried out by our robots in its current state due to the lack of adequate

safety measures. The installation does not include a robust control system with properly integrated passive and active elements, as required to form a functional Flexible Manufacturing Cell (FMC).

If it was to be implemented, it should include measures that protect workers and equipment from being damaged.

Project Specifications - B

B - Functional summary

To briefly describe the functional process of the program execution we are going to divide it into several sections.

By one hand, we have designed every smart component as a smart module for each object in the program. We have used in each sources, linear movers and different logic to prevent the objects moving when the robot has touched or initialized the process with them.

Therefore, we can easily work with the objects generated by sensor planes and directly using different signals for processing while at the same time we keep it all clean and organized for understanding.

Moreover, we have used input and output signals in both robots used, so we can synchronize each of them and with the different components of their environment, allowing us then, to create several lines with the same systems so we could simulate more complex process keeping an easily and understanding view of the processing of the program.

Robots:

Our project involves the use of two collaborative robots, each meeting the minimum objectives required for an industrial robotic system. Both robots are equipped with two defined Tool Center Points (TCPs): one named grip, used for picking and placing components, and the other named pen, designated for welding operations.

Additionally, two distinct Work Objects (Wobj) named Table have been created, one for each robot. These Wobj are strategically positioned at opposite ends of the

solar panel assembly area to provide accurate spatial references for the welding process. This configuration ensures that each robot operates within its own defined coordinate system, allowing for better precision and synchronization during joint tasks such as positioning and welding.

Regarding tools:

In our project, we did not create a completely new tool from primitives or by importing a graphical model. Instead, we adapted an existing tool named pen by assigning it a different functional role. Originally a generic tool, we redefined its purpose within the program to represent a welding tool. This was achieved by configuring its behavior and integrating it into the welding sequence using the appropriate tooldata settings and RAPID instructions.

By doing this, we were able to simulate welding operations effectively

Optimization of procedures:

Procedural optimization in our program is achieved through the use of variables such as tooldata and wobjdata, which allow for greater flexibility, code reuse, and system adaptation without the need to manually modify each instruction. By defining the tool (tooldata) in a parameterized manner, the robot can automatically adjust its behavior based on the active tool, facilitating the transition between gripping and welding operations. Similarly, the use of wobjdata allows for the definition of specific coordinate systems for each work zone, simplifying the programming of relative trajectories and ensuring accuracy in variable environments. This approach reduces errors, improves code maintainability, and allows the system to be adapted with minimal changes if the physical layout or tools vary.

Digital Signals

In our program, we use digital input and output signals to coordinate the robot's interaction with the environment. The digital outputs are primarily used to control robot elements, such as activating the gripping tool.

On the other hand, the digital inputs are used primarily for synchronization between the two robots, allowing one to wait for the other's signal before continuing the task.

Youtube Video

<https://www.youtube.com/watch?v=YyQ8TbGmiWs>

[IRB 2600 for OmniCore, Product specification](#)

[Operating manual - RobotStudio](#)